Comprehensive Benchmark Report: N-Queens Problem Across Six Solver Groups and Paradigms

Nguyễn Khả Đạt[a]

[a]*Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam*

ARTICLE INFO

Keywords:
N-Queens
Propositional Satisfiability
Constraint Programming
Mixed-Integer Programming
Benchmarking
PySAT

ABSTRACT

This report presents an empirical evaluation of solving the classic N-Queens problem across six major solver groups and modeling paradigms: Propositional Satisfiability (SAT with CaDiCaL, Glucose3, MiniSat22, Kissat, and MapleCM), Constraint Programming via CP-SAT (OR-Tools), Mixed-Integer Programming (MIP with CBC, SCIP), High-Performance Commercial MIP (Gurobi), Commercial CPLEX MIP, and Commercial CPLEX CP. We evaluate encoding efficiency, variable scaling, constraint footprint, model construction overhead, and execution times across evaluated board sizes up to $N = 128$ under standardized 3-repetition benchmark runs with hard timeouts. Detailed performance metrics and visualizations highlight the trade-offs between expressive global constraints and low-level clausal representations, alongside timeout analyses across evaluated instances, CP solve overhead management, and solution status semantics (FEASIBLE vs OPTIMAL).

1. Introduction

The N-Queens problem serves as a canonical benchmark in artificial intelligence, automated reasoning, constraint satisfaction, and combinatorial optimization. Placing N non-attacking queens on an $N \times N$ chessboard tests the scalability, constraint propagation, and search pruning power of modern discrete solvers.

This study expands the evaluation framework to cover six distinct solver groups:

- **SAT Solvers**: CaDiCaL and Kissat [1], Glucose3 [2], MiniSat22 [3], and MapleCM (based on MapleCOMSPS [4]) evaluated across Pairwise, Binary, Sequential Counter, Commander, and Product cardinality encodings (implemented using the PySAT toolkit [5]).
- **Constraint Programming (CP-SAT)**: OR-Tools CP-SAT using Integer, Boolean, and Allowed assignments configurations [9].
- **Open-Source MIP**: CBC [10] and SCIP [11] solvers using Binary and Integer linear formulations.
- **Commercial MIP (Gurobi)**: High-performance Gurobi optimization engine [12] with advanced presolve and branch-and-cut routines (noting size-limited (restricted) license bounds at $N \leq 44$).
- **Commercial CPLEX MIP**: IBM ILOG CPLEX Optimizer using the Mathematical Programming (MIP) engine [13].
- **Commercial CPLEX CP**: IBM ILOG CPLEX Optimizer using the Constraint Programming (CP) engine [14], scaling up to $N = 128$.

2. Encodings and Mathematical Formulations

Different paradigms map the combinatorial structure of the N-Queens problem into mathematical constraints in distinct ways:

24021404@vnu.edu.vn (N.K. Đạt)
ORCID(s):

[1]Course: Modern Topics in Computer Science (2627I_INT3011E_4) – Student ID: 24021404

Table 1
Asymptotic Size of the SAT Encodings

Encoding	Clauses per Group (Size n)	Aux. Variables per Group	Total Clauses	Total Aux. Variables
Pairwise	$\binom{n}{2} = \frac{n(n-1)}{2}$	0	$\Theta(N^3)$	0
Binary	$n\lceil \log_2 n \rceil$	$\lceil \log_2 n \rceil$	$\Theta(N^2 \log N)$	$\Theta(N \log N)$
Sequential	$3n-4$	$n-1$	$\Theta(N^2)$	$\Theta(N^2)$
Commander	$\Theta(n)$	$\Theta(n)$	$\Theta(N^2)$	$\Theta(N^2)$
Product	$\Theta(n)$	$\Theta(\sqrt{n})$	$\Theta(N^2)$	$\Theta(N^{1.5})$

2.1. Propositional Satisfiability (SAT) Encodings

Cardinality and at-most-one encodings into CNF are surveyed in [15]. Boolean decision variables $x_{i,j}$ denote whether a queen is placed at row i, column j ($1 \le i, j \le N$), requiring N^2 primary decision variables. As implemented in `nqueens_encoding` (`src/encodings.py`), the problem is formulated via four distinct constraint families:

1. **At-Least-One (ALO) per row**: $\bigvee_{j=1}^{N} x_{i,j}$ for each row $1 \le i \le N$, contributing N clauses.
2. **At-Most-One (AMO) per row**: $\mathrm{AMO}(x_{i,1}, \ldots, x_{i,N})$ for each row $1 \le i \le N$ (N groups of size N).
3. **At-Most-One (AMO) per column**: $\mathrm{AMO}(x_{1,j}, \ldots, x_{N,j})$ for each column $1 \le j \le N$ (N groups of size N).
4. **At-Most-One (AMO) per diagonal**: $\mathrm{AMO}(x_{i,j} : (i,j) \in \mathrm{diag})$ for each major and minor diagonal of length $k \ge 2$, comprising $2(2N-3)$ groups of sizes $k \in \{2, \ldots, N\}$.

Crucially, all three AMO constraint families (rows, columns, and diagonals) invoke the identical modular AMO encoding function. Five representative AMO encodings are evaluated:

- **Pairwise**: Adds negative binary clauses $(\neg u \vee \neg v)$ for each distinct pair. Per AMO group of size n, it generates $\binom{n}{2} = \frac{n(n-1)}{2}$ clauses without auxiliary variables. Summed over all rows, columns, and diagonals, the total clause count is $2N\binom{N}{2} + 2\left(2\binom{N}{3} + \binom{N}{2}\right) + N = \frac{5}{3}N^3 - 2N^2 + \frac{4}{3}N = \Theta(N^3)$ (yielding 428,800 clauses at $N = 64$ and 3,462,656 at $N = 128$).
- **Binary (Bitwise)**: Assigns binary index vectors to literals using $\lceil \log_2 n \rceil$ auxiliary variables and $n\lceil \log_2 n \rceil$ clauses per group of size n, resulting in $\Theta(N^2 \log N)$ total clauses and $\Theta(N \log N)$ auxiliary variables.
- **Sequential Counter (Sinz)** [6]: Chains partial sums with $n-1$ auxiliary variables and $3n-4$ clauses per group ($n \ge 2$), achieving $\Theta(N^2)$ total clauses and $\Theta(N^2)$ auxiliary variables.
- **Commander (Klieber & Kwon)** [7]: Recursively partitions literals into blocks of size 3 and elects commander variables, posting $\Theta(n)$ clauses and $\Theta(n)$ auxiliary variables per group, yielding $\Theta(N^2)$ total clauses and $\Theta(N^2)$ auxiliary variables.
- **Product (Chen)** [8]: Embeds literals into a 2D grid of size $p \times q$ with $p = \lceil \sqrt{n} \rceil$ and $q = \lceil n/p \rceil$, enforcing binary AMO on row and column variables alongside $2n$ assignment clauses, yielding $\Theta(n)$ clauses and $\Theta(\sqrt{n})$ auxiliary variables per group ($\Theta(N^2)$ total clauses and $\Theta(N^{1.5})$ auxiliary variables).

2.2. Constraint Programming (CP-SAT) & Global Constraints

OR-Tools CP-SAT utilizes integer decision variables $q_i \in \{0, 1, \ldots, N-1\}$ representing the column of the queen in row i. Global constraints enforce:

1. $\mathrm{AllDifferent}(q_1, \ldots, q_N)$ (columns)
2. $\mathrm{AllDifferent}(q_1 + 1, q_2 + 2, \ldots, q_N + N)$ (main diagonals)
3. $\mathrm{AllDifferent}(q_1 - 1, q_2 - 2, \ldots, q_N - N)$ (anti-diagonals)

This formulation utilizes N primary integer decision variables, alongside auxiliary variables introduced during model construction (resulting in 383 variables for $N = 128$ in the integer configuration proto), and 3 global constraints, pruning infeasible branches effectively.

Table 2
Software and Solver Package Versions

Package / Tool	Version	Solvers / Modules Included
PySAT	1.9.dev15	CaDiCaL 1.5.3, Kissat 4.0.4, Glucose 3.0, MiniSat 2.2, MapleCM
OR-Tools	9.15.6755	CP-SAT solver, Linear Solver (CBC, SCIP interfaces)
Gurobi (`gurobipy`)	13.0.3	Gurobi Optimizer (size-limited (restricted) license)
IBM ILOG CPLEX (`cplex`)	22.2.0.1	CPLEX Mathematical Programming (MIP) engine
DOcplex (`docplex`)	2.32.264	CPLEX CP Optimizer API (`cpoptimizer`)
Python	3.14.7	Execution runtime (virtualenv `SAT`, macOS 15.7.4)

2.3. Mixed-Integer Programming (MIP)

MIP models formulate the problem as linear inequalities over binary variables $x_{i,j} \in \{0,1\}$ or integer variables $q_i \in \{0,1,\ldots,N-1\}$, coupled with linear diagonal separation cuts:

$$\sum_{i=1}^{N} x_{i,j} = 1 \quad (\forall j), \quad \sum_{j=1}^{N} x_{i,j} = 1 \quad (\forall i), \quad \sum_{(i,j)\in \mathrm{diag}} x_{i,j} \leq 1$$

3. Experimental Setup

3.1. Hardware and Software Environment

All benchmarks were conducted on a MacBook Air equipped with an Apple M4 processor and 16 GB of unified memory, operating under macOS 15.7.4 (Sequoia). The evaluation runtime was isolated within a dedicated Python 3.14.7 virtual environment (`SAT`). Table 2 lists the exact software library and solver package versions employed in the reference benchmark. The SAT solvers are directly bundled within PySAT 1.9.dev15 as compiled C/C++ libraries, corresponding to the concrete solver classes instantiated in `src/sat_model.py`: CaDiCaL 1.5.3 (`Cadical153`), Kissat 4.0.4 (`Kissat404`), Glucose 3.0 (`Glucose3`), MiniSat 2.2 (`Minisat22`), and MapleCOMSPS/MapleCM (`MapleCM`).

Solver configurations, worker allocations, and timeout parameters are specified as follows:

- **SAT Solvers (PySAT)**: Solvers run as single-threaded C++ processes using deterministic default search heuristics without explicit random seeds. For solvers with cooperative interrupt support (`Glucose3`, `Minisat22`, `MapleCM`), internal soft timeouts are handled via a Python `threading.Timer` invoking `solve_limited(expect_interrupt=True)`. Solvers without native interrupt mechanisms in PySAT (`Cadical153`, `Kissat404`) rely on parent process hard timeouts.
- **OR-Tools CP-SAT**: Executes with default multithreaded worker allocation (the number of search workers is not explicitly pinned in code, adhering to OR-Tools heuristics based on host hardware; thus solving efficiency reflects available CPU cores), default random seed, and remaining time budget passed via `parameters.max_time_in_seconds`.
- **MIP Solvers (CBC, SCIP)**: Invoked through OR-Tools `pywraplp.Solver` with default single-threaded parameters and time limits specified via `set_time_limit()` in milliseconds.
- **Gurobi**: Commercial solver Gurobi operated under size-limited (restricted) license limitations ($N \leq 44$) with default multithreading (`Threads = 0`), default random seed (`Seed = 0`), and remaining time budget forwarded via `Params.TimeLimit`.
- **CPLEX MIP and CP Optimizer**: CPLEX operated under full license support allowing scaling up to $N = 128$. CPLEX MIP uses default thread allocation and random seeds, with time limits passed via `set_time_limit()`. CP Optimizer (`cplex-cp`) executes via the external `cpoptimizer` binary with default worker allocation, default seed, `LogVerbosity = Quiet`, and time limits configured via `params["TimeLimit"]`.

3.2. Benchmark Instances and Scaling

Evaluated board sizes span the progression $N \in \{8, 16, 32, 44, 64, 128\}$. The committed benchmark dataset comprises 170 distinct configuration records, where each record represents a unique combination of $(N, \text{model variant}, \text{solver})$ evaluated over 3 complete repetitions, rather than an exhaustive Cartesian product across all solvers and scales. Open-source MIP solvers (CBC, SCIP) were evaluated up to $N = 64$, whereas SAT, CP-SAT, and CPLEX configurations scaled up to $N = 128$.

The intermediate instance size $N = 44$ was evaluated exclusively for Gurobi to analyze solver behavior at the boundary of its license capacity: the restricted license supports at most 2,000 variables and 2,000 linear constraints [16]. Under the binary matrix formulation with N^2 binary decision variables and row, column, and diagonal cuts, $N = 44$ produces $44^2 = 1,936$ variables ($\leq 2,000$) and 258 linear constraints ($\leq 2,000$). At $N = 45$, the model requires $45^2 = 2,025$ variables, and at $N = 64$ requires 4,096 variables; because both exceed the 2,000-variable limit, these dimensions were not evaluated with Gurobi.

3.3. Reproducibility and Metric Definitions

Each benchmark run executes inside an isolated Python child process with a strict 30.0-second timeout across 3 repetitions without early termination. Benchmarking runs can be reproduced using:

```
python src/benchmark.py -time-limit 30.0 -repetitions 3 -output-dir results
```

Aggregated metrics, comparison tables, and report figures are generated via:

```
python src/generate_report.py
```

Because multithreaded solvers (CP-SAT, Gurobi, CPLEX) utilize host-dependent thread pools, exact execution runtimes depend on host CPU architecture, clock frequencies, and core availability. The benchmark records the following primary metrics:

- `import_time`: Duration (seconds) to load paradigm and solver modules in the child process prior to model building.
- `build_time`: Time (seconds) elapsed during variable instantiation and constraint posting.
- `solve_time`: Active execution duration (seconds) spent inside the solver's solve method.
- `model_time`: Sum of `build_time + solve_time` per repetition (reported as the median of per-run sums), isolating model construction and solving performance from module import latency.
- `time_seconds`: Total child process phase duration (`import_time + build_time + solve_time`), capped at 30.0 s upon timeout.
- `wall_time`: Parent-measured wall-clock elapsed time (via `time.perf_counter()`) spanning child process launch, IPC, and termination.
- `runs_total`, `runs_success`, `runs_timeout`, `runs_error`: Granular counters tracking outcomes across all 3 repetitions to ensure failure and timeout distributions are captured without masking.

4. Empirical Results and Performance Analysis

4.1. SAT Solvers Performance

Table 3 summarizes SAT benchmark results across diverse encodings and solvers (total time in seconds: import + build + solve).

Table 4 presents the model size (variable and clause counts) across board sizes N and encoding variants.

4.2. CP-SAT Performance

Table 5 outlines OR-Tools CP-SAT performance.

4.3. Open-Source MIP Performance

Table 6 summarizes CBC and SCIP performance.

Table 3
SAT Solvers Performance Metrics (Total Time in Seconds: Import + Build + Solve)

N	Encoding	CaDiCaL	Glucose3	MiniSat22	Kissat	MapleCM
8	binary	0.011	0.011	0.011	0.015	0.011
8	commander	0.010	0.010	0.010	0.013	0.010
8	pairwise	0.011	0.011	0.011	0.015	0.011
8	product	0.011	0.011	0.011	0.014	0.011
8	sequential	0.010	0.011	0.011	0.012	0.011
16	binary	0.020	0.099	0.014	0.030	0.020
16	commander	0.016	0.027	0.013	0.033	0.050
16	pairwise	0.015	0.013	0.020	0.033	0.016
16	product	0.018	0.081	0.013	0.031	0.031
16	sequential	0.022	0.025	0.013	0.035	0.021
32	binary	0.040	0.288	0.019	0.049	0.416
32	commander	0.073	1.390	0.095	0.187	0.406
32	pairwise	0.036	0.032	0.023	0.079	0.057
32	product	0.043	0.775	0.031	0.053	0.608
32	sequential	0.054	0.181	0.019	0.093	1.292
64	binary	0.297	1.452	0.088	0.867	0.515
64	commander	0.347	T/O	0.241	0.403	1.006
64	pairwise	0.179	0.226	0.111	0.639	0.719
64	product	0.975	T/O	0.280	0.420	1.261
64	sequential	1.214	T/O	0.171	0.092	9.945
128	binary	1.370	T/O	0.193	2.323	T/O
128	commander	0.399	T/O	T/O	1.434	T/O
128	pairwise	1.553	8.143	1.106	5.931	1.740
128	product	2.220	T/O	T/O	1.356	25.999
128	sequential	8.637	T/O	T/O	1.385	T/O

4.4. Commercial Gurobi and CPLEX Performance

Tables 7 and 8 present Gurobi and CPLEX results.

5. Timeout Analysis

Table 9 details the timeout counts encountered across solver paradigms and board sizes under the strict 30-second limit.

6. Visualizations

Table 4
SAT Model Size Metrics by Encoding Variant

N	Encoding	Variables	Clauses
8	binary	174	720
8	commander	146	528
8	pairwise	64	736
8	product	412	880
8	sequential	274	596
16	binary	572	3912
16	commander	682	2610
16	pairwise	256	6336
16	product	1188	3208
16	sequential	1186	2716
32	binary	1850	19768
32	commander	2956	11326
32	pairwise	1024	52608
32	product	3740	13036
32	sequential	4930	11564
64	binary	6136	95512
64	commander	11958	47096
64	pairwise	4096	428800
64	product	11064	46944
64	sequential	20098	47692
128	binary	21238	447704
128	commander	48836	192754
128	pairwise	16384	3462656
128	product	35598	182888
128	sequential	81154	193676

Table 5
CP-SAT Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	allowed	ortools-cpsat	8	29	0.183747
8	boolean	ortools-cpsat	64	46	0.179495
8	integer	ortools-cpsat	23	3	0.180548
16	allowed	ortools-cpsat	16	121	0.214220
16	boolean	ortools-cpsat	256	94	0.202742
16	integer	ortools-cpsat	47	3	0.183573
32	allowed	ortools-cpsat	32	497	0.410215
32	boolean	ortools-cpsat	1024	190	0.204157
32	integer	ortools-cpsat	95	3	0.216879
64	allowed	ortools-cpsat	64	2017	3.249208
64	boolean	ortools-cpsat	4096	382	0.280460
64	integer	ortools-cpsat	191	3	0.272827
128	allowed	ortools-cpsat	-	-	T/O
128	boolean	ortools-cpsat	16384	766	0.720303
128	integer	ortools-cpsat	383	3	0.647104

Table 6
MIP (CBC/SCIP) Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	binary	cbc	64	46	0.038824
8	binary	scip	64	46	0.040379
8	integer	cbc	72	54	0.037656
8	integer	scip	72	54	0.040296
16	binary	cbc	256	94	0.055535
16	binary	scip	256	94	0.041167
16	integer	cbc	272	110	0.049211
16	integer	scip	272	110	0.041906
32	binary	cbc	1024	190	0.087756
32	binary	scip	1024	190	0.727264
32	integer	cbc	1056	222	0.085858
32	integer	scip	1056	222	0.702270
64	binary	cbc	4096	382	0.342792
64	binary	scip	4096	382	1.336934
64	integer	cbc	4160	446	0.432027
64	integer	scip	4160	446	3.569228

Table 7
Gurobi Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	binary	gurobi	64	42	0.008870
16	binary	gurobi	256	90	0.009543
32	binary	gurobi	1024	186	0.019588
44	binary	gurobi	1936	258	0.013442

Table 8
CPLEX Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	binary	cplex	64	42	0.164246
8	integer	cplex-cp	8	3	0.161112
16	binary	cplex	256	90	0.163949
16	integer	cplex-cp	16	3	0.161521
32	binary	cplex	1024	186	0.167906
32	integer	cplex-cp	32	3	0.164400
64	binary	cplex	4096	378	0.183581
64	integer	cplex-cp	64	3	0.164295
128	binary	cplex	16384	762	0.244642
128	integer	cplex-cp	128	3	0.169599

Table 9
Timeout Analysis Across Solvers and Board Sizes

Paradigm	Solver	Board Size (N)	Timeout Count
CP-SAT	ortools-cpsat	128	1
SAT	glucose3	64	3
SAT	glucose3	128	4
SAT	maplecm	128	3
SAT	minisat22	128	3

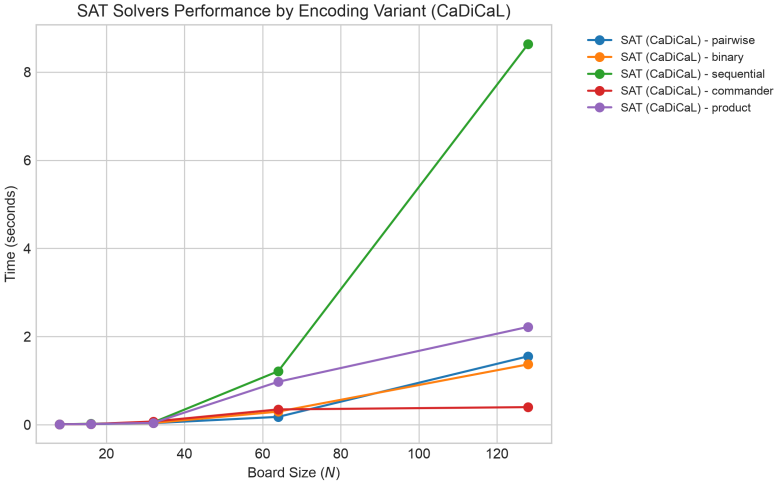


Figure 1: SAT (CaDiCaL) total time scaling across board sizes N.

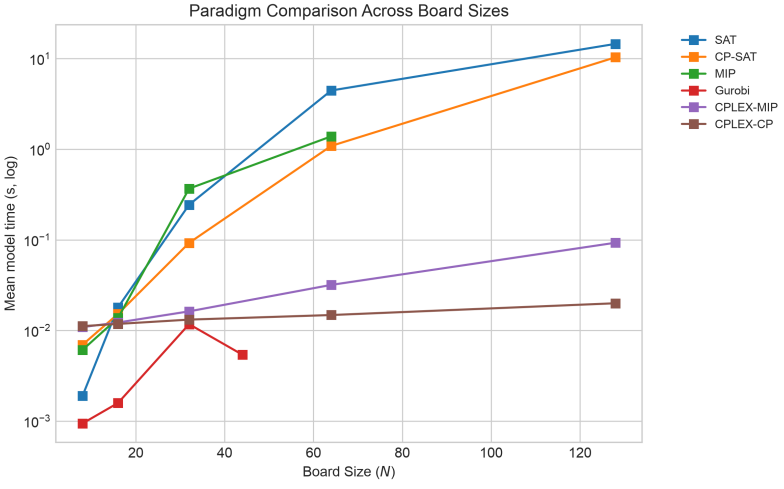


Figure 2: Comparison of mean model time across all solver paradigms (build + solve; log scale; timeouts counted as 30 s, so means are lower bounds).

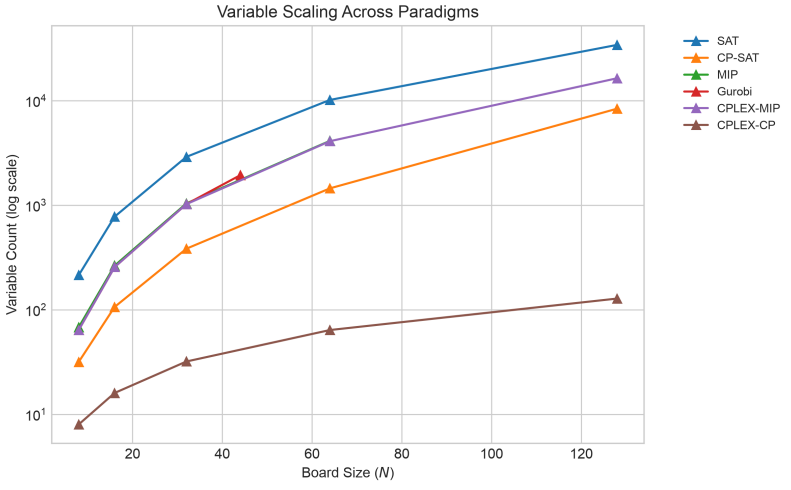


Figure 3: Variable count scaling across different solving paradigms and solvers.

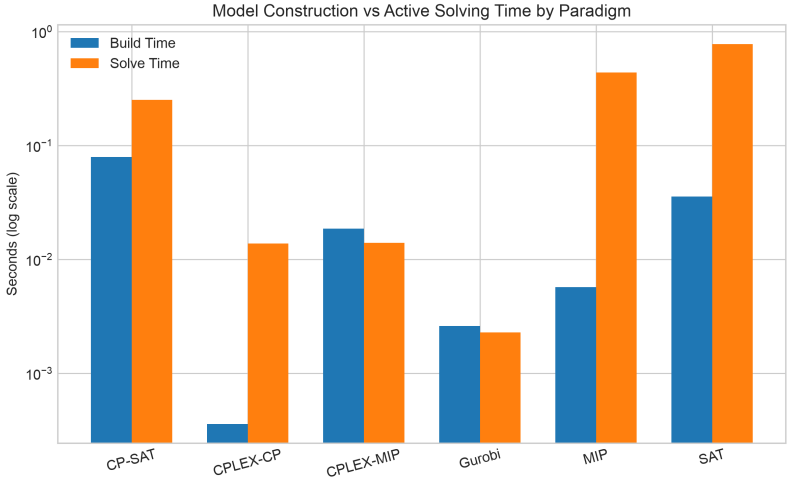


Figure 4: Model construction overhead versus active solving time by paradigm.

7. Discussion and Comprehensive Solver Comparison

The empirical results reveal distinct performance characteristics and limitations across all six solver groups (SAT, CP-SAT, open-source MIP, Gurobi, CPLEX MIP, and CPLEX CP) when scaling across the N range ($N = 8$ to 128):

- **Comparative Performance Across Six Solver Groups**: Across moderate to large scales ($N = 8$ to 128), commercial engines (CPLEX MIP, CPLEX CP, Gurobi) and high-performance solvers (CP-SAT, CaDiCaL) demonstrate exceptional scalability. At $N = 128$, recorded runtimes range from approximately 169.6 ms for CPLEX CP to 0.647 s for CP-SAT (noting that measurement methodologies differ across solvers, where reported `time_seconds` is the median over repetitions of each run's `import_time + build_time + solve_time` — with build and solve medians reported separately so they may not sum exactly — excluding parent-side process launch overhead; similarly, `model_time` represents the median across repetitions of per-run `build_time + solve_time` values, which need not equal `median(build_time) + median(solve_time)`). Crucially, `import_time` is library-dependent and constant with respect to N (for instance, OR-Tools CP-SAT incurs ≈ 0.18 s of Python module import overhead regardless of N), meaning cross-paradigm runtime comparisons at small N primarily reflect library import latency rather than algorithmic solving speed. To isolate solver efficiency from module import overhead, Figure 2 (Plot 2) evaluates *model time* (`build_time + solve_time`), with timeouts counted as 30 s, whereas many SAT combinations take tens of seconds or timeout.

- **FEASIBLE vs. OPTIMAL Status Semantics**: For pure satisfaction problems (such as N-Queens without an objective function), finding any valid non-attacking configuration successfully solves the problem. Solution status labels are solver-specific and API-dependent: CPLEX CP reports `FEASIBLE`, whereas CP-SAT reports `OPTIMAL` and MIP solvers may report `OPTIMAL` for satisfaction models. Solution validity in our framework is independently verified using the benchmark's verification module rather than relying solely on solver status strings.

- **CP Optimizer Process Startup Overhead**: CP Optimizer relies on an external binary executable (`cpoptimizer`). Spawning this subprocess incurs inter-process communication and process startup overhead; actual total measured runtimes for CPLEX CP increase from approximately 161.1 ms at $N = 8$ to 169.6 ms at $N = 128$. Here, `solve_time` measures the wall-clock duration around the solve call (including cpoptimizer process startup), while `solver_reported_time` is the number reported by the solver (median of 10.0 ms at $N = 128$); thus, measured runtimes include process startup and inter-process communication, and should not be directly compared on equal footing with CP-SAT's wall-clock measurement.

- **Gurobi Size-Limited (Restricted) License Limitations**: Gurobi's size-limited (restricted) license restricts model dimensions for larger instances, capping instances at $N \leq 44$. For $N > 44$, full commercial licenses or alternative solvers (CPLEX, CP-SAT, CaDiCaL) are required.

- **SAT Scaling and Encodings**: CaDiCaL and Kissat handle large clause databases efficiently up to $N = 128$ (with maximum recorded runtimes of approximately 8.64 s for CaDiCaL and 5.93 s for Kissat), whereas MapleCM reaches a maximum recorded runtime of approximately 26.00 seconds under product encoding at $N = 128$, and Glucose3, MiniSat22, and MapleCM encounter frequent timeouts (10/25 cases timed out at $N = 128$, specifically Glucose3: 4, MiniSat22: 3, MapleCM: 3). Compact cardinality encodings such as Commander (47,096 clauses at $N = 64$) and Sequential Counter (47,692 clauses at $N = 64$) dramatically reduce clause footprints compared to naive Pairwise encodings (428,800 clauses at $N = 64$).

- **CP-SAT Allowed Assignment Timeout at** $N = 128$: The `allowed` assignment variant posts $\binom{N}{2}$ pairwise binary table constraints whose cumulative allowed tuple count scales as $\mathcal{O}(N^4)$, surging from 7.96 million tuples at $N = 64$ to 130.7 million at $N = 128$. Measured build and solve times at $N = 32$ and $N = 64$ scale by approximately 16× (0.060 s to 0.963 s) and 12× (0.169 s to 2.105 s) per doubling of N. Extrapolating these scaling trends from $N = 32$ and $N = 64$ (rather than direct measurement at $N = 128$) projects an estimated build time of ~ 15.6 s and solve time of ~ 26.2 s (totalling ~ 41.8 s,

exceeding the 30 s limit), indicating that the timeout is likely driven by table constraint size rather than search difficulty.

8. Conclusion

This benchmark evaluation demonstrates that while modern SAT, CP-SAT, and commercial CP/MIP engines solve moderate instances efficiently, performance varies significantly at $N = 128$. Performance is heavily dictated by solver sensitivity, global constraint propagation, and encoding efficiency: CaDiCaL, Kissat, CPLEX CP, and CP-SAT handle large scales effectively, whereas clausal blow-up and solver heuristics cause frequent timeouts for older SAT engines.

References

[1] A. Biere, K. Fazekas, M. Fleury, and M. Heisinger, "CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020," in *Proceedings of SAT Competition 2020 – Solver and Benchmark Descriptions*, T. Balyo, N. Froleyks, M. Heule, M. Iser, M. Järvisalo, and M. Suda, Eds., Department of Computer Science Report Series B, vol. B-2020-1, pp. 51–53, University of Helsinki, 2020.

[2] G. Audemard and L. Simon, "Predicting Learnt Clauses Quality in Modern SAT Solvers," in *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009)*, pp. 399–404, 2009.

[3] N. Eén and N. Sörensson, "An Extensible SAT-solver," in *Theory and Applications of Satisfiability Testing (SAT 2003)*, Lecture Notes in Computer Science, vol. 2919, pp. 502–518, Springer, 2004.

[4] J. H. Liang, V. Ganesh, P. Poupart, and K. Czarnecki, "Learning Rate Based Branching Heuristic for SAT Solvers," in *Theory and Applications of Satisfiability Testing – SAT 2016*, Lecture Notes in Computer Science, vol. 9710, pp. 123–140, Springer, 2016.

[5] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," in *International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 428–437, 2018.

[6] C. Sinz, "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints," in *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827–831, 2005.

[7] W. Klieber and G. Kwon, "Efficient CNF Encoding for Selecting 1 from n Objects," in *Fourth Workshop on Constraints in Formal Verification (CFV)*, 2007.

[8] J. Chen, "A New SAT Encoding for the At-Most-One Constraint," in *Proceedings of the 9th International Workshop on Constraint Modelling and Reformulation (ModRef)*, 2010.

[9] L. Perron and V. Furnon, "OR-Tools," version 9.15, Google. Available: https://developers.google.com/optimization (accessed 19 October 2026).

[10] J. Forrest and R. Lougee-Heimer, "CBC User Guide," in *Emerging Theory, Methods, and Applications*, INFORMS, pp. 257–277, 2005. Available: https://github.com/coin-or/Cbc (accessed 19 October 2026).

[11] K. Bestuzheva et al., "The SCIP Optimization Suite 8.0," Technical Report, ZIB-Report 21-41, Zuse Institute Berlin, 2021.

[12] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," version 13.0.3, 2026. Available: https://docs.gurobi.com/ (accessed 19 October 2026).

[13] IBM Corp., "IBM ILOG CPLEX Optimization Studio CPLEX User's Manual," version 22.2.0.1, 2026. Available: https://www.ibm.com/docs/en/icos (accessed 19 October 2026).

[14] P. Laborie, J. Rogerie, P. Shaw, and P. Vilím, "IBM ILOG CP Optimizer for Scheduling," *Constraints*, vol. 23, no. 2, pp. 210–250, 2018.

[15] A. Biere, M. Heule, H. van Maaren, and T. Walsh, *Handbook of Satisfiability*, 2nd ed. IOS Press, 2021.

[16] Gurobi Optimization, LLC, "How do I resolve a 'Model too large for size-limited Gurobi license' error?", *Gurobi Help Center*. Available: https://support.gurobi.com/hc/en-us/articles/360051597492 (accessed 16 October 2026).

CRediT Authorship Contribution Statement

Nguyễn Khả Đạt: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing - original draft, Writing - review & editing.

Declaration of Competing Interest

The author declares that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation

During the preparation of this work, the author used Claude (Anthropic) for manuscript review and text editing, and Gemini (Google) for code drafting and testing. All experimental evaluations were executed independently by the author on personal hardware, and the author reviewed, verified, and takes full responsibility for the contents of this publication.

Data Availability

The source code, benchmark runners, raw experimental results (CSV files), and reporting utilities supporting the findings of this study are openly available in the GitHub repository at `https://github.com/datnk0116/Nqueen_KHMT`.